

2

EC2 servers

2

Availability Zones

5

Websites, one proxy

3

CloudWatch alarms

FINAL PROJECT PRESENTATION



HIGH AVAILABILITY NGINX PLATFORM

Reverse proxy web application on AWS, built with Terraform, Ansible and CloudWatch

Cloud / DevOps project



INTRODUCTION



A highly available web platform on AWS.

Five web applications are served through an Nginx reverse proxy on two EC2 instances, behind an Application Load Balancer.

Terraform builds the infrastructure, Ansible configures the servers, and CloudWatch watches over the result. Reverse proxy helps to improve the security, performance and scalability.

REGION

ap-south-1

COMPUTE

2 × EC2, Ubuntu 22.04

LOAD BALANCER

ALB, health check /health

WEBSITES

5 behind one Nginx proxy

MONITORING

CloudWatch Alarms & Dashboard

PROBLEM STATEMENT



Running several web applications on one server tier creates three recurring problems.

SINGLE POINT OF FAILURE

One server or one zone going down takes every application offline.

MANUAL, INCONSISTENT SETUP

Hand-built servers drift apart and are slow and risky to rebuild.

NO EARLY WARNING

Without health checks and alarms, users report outages before the team knows.

OBJECTIVES



Automate the infrastructure

Terraform init, validate, plan and apply create the whole AWS environment in a faster, repeatable way.



Reduce manual work

Ansible installs and configures Nginx and the websites, so every server ends up identical.



Track application health

CloudWatch alarms detect and report unhealthy servers quickly.

SCOPE



IN SCOPE

- AWS network: VPC, 2 public subnets, Internet Gateway
- 2 EC2 servers and an ALB across 2 Availability Zones
- Nginx reverse proxy for 5 websites
- 3 CloudWatch alarms and an SNS topic

NEXT STEPS

- HTTPS listener with a certificate
- Auto Scaling for capacity and replacement
- Private subnets behind a NAT gateway
- CI/CD pipeline and Docker containers

METHODOLOGY



Everything is code, so everything is repeatable.

1

TERRAFORM

Infrastructure as Code

VPC, subnets, security groups, EC2, ALB, alarms are described in .tf files and created with one command.

2

ANSIBLE

Configuration as Code

Nginx, the reverse-proxy rules and the five websites are applied by a playbook, and can be re-applied at any time.

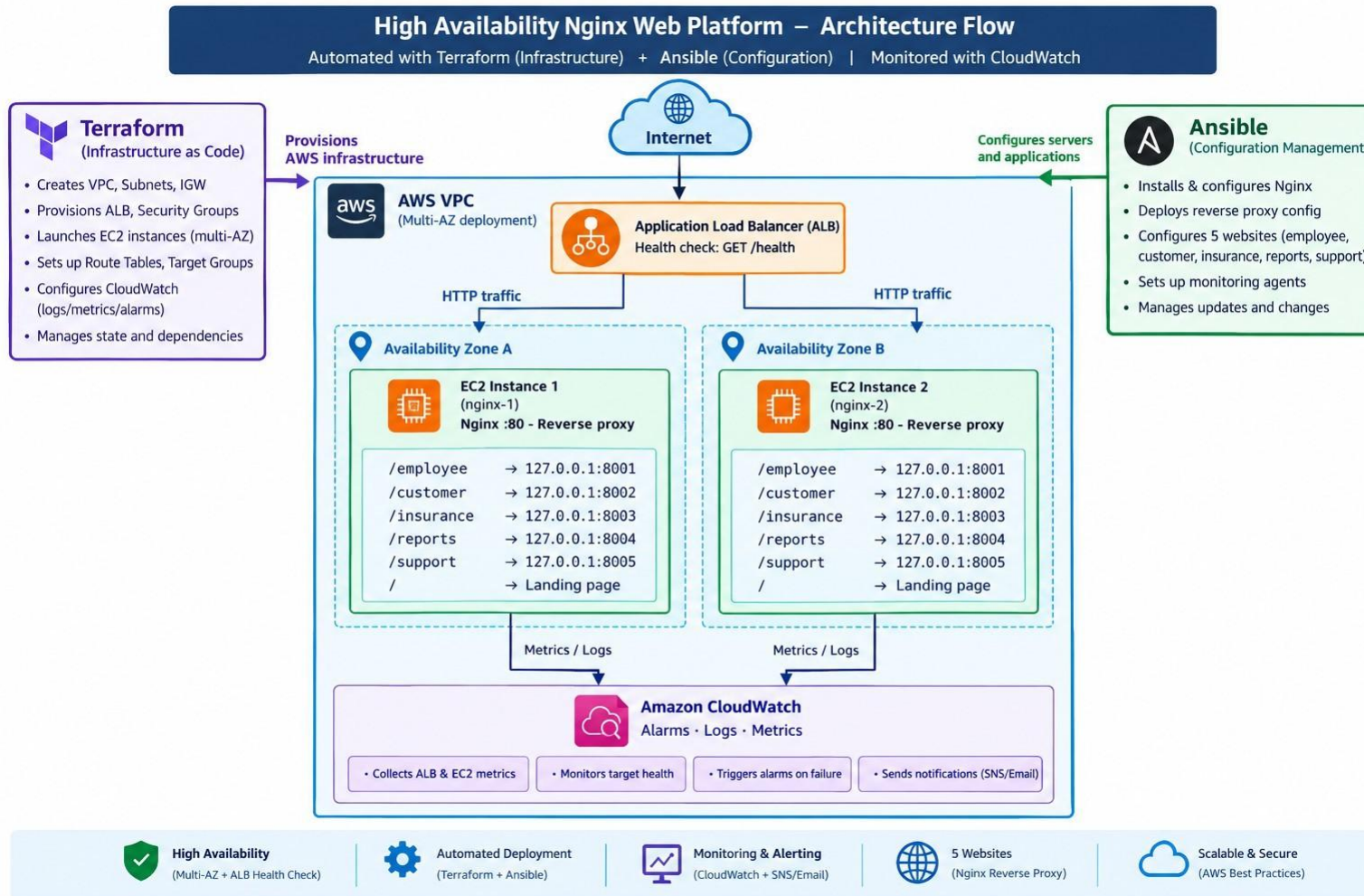
3

CLOUDWATCH

Monitoring as Code

Health alarms are created by Terraform, so monitoring exists from the first deployment.

ARCHITECTURE



Terraform provisions

VPC, ALB, 2 EC2 servers, security groups, alarms

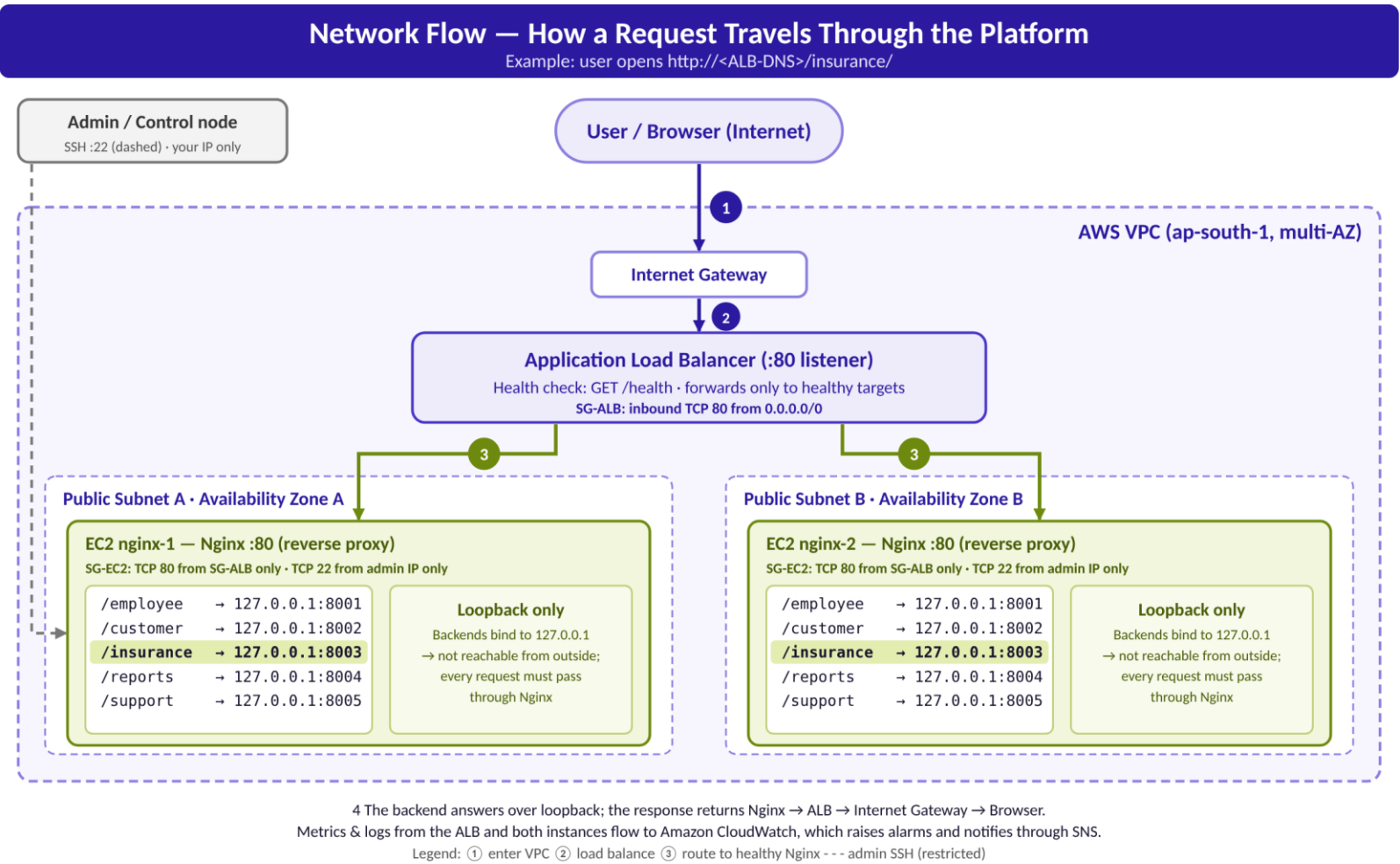
Ansible configures

Nginx, reverse-proxy rules, five websites

CloudWatch watches

ALB and EC2 health, alerts through CloudWatch dashboards & SNS

NETWORK FLOW



ALB

Port 80 open to the internet

EC2 SERVERS

Port 80 from the ALB only

SSH

Port 22 from your IP only

BACKENDS

Listen on 127.0.0.1 only

PROJECT IMPLEMENTATION



1

Provision

Terraform

```
terraform init
terraform validate
terraform plan
terraform apply
```

2

Configure

Ansible

```
ansible nginx_servers -m
ping
ansible-playbook site.yml
```

3

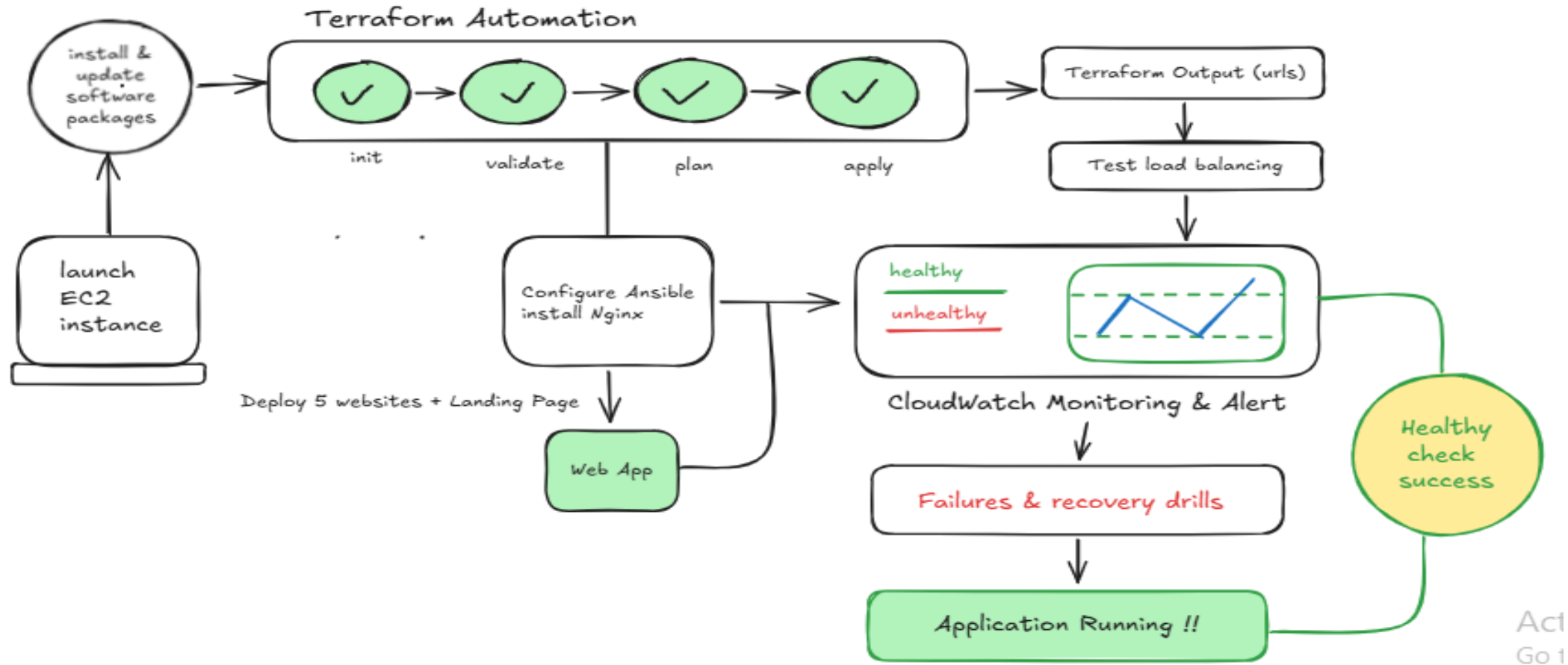
Verify

Browser and curl

```
terraform output
websites_urls

curl -sI http://$ALB/ |
grep -i x-served-by
```

DEPLOYMENT WORKFLOW



PROJECT STRUCTURE



```
ha-nginx-platform/  
├── terraform/  
│   ├── providers.tf, variables.tf  
│   ├── vpc.tf, security-groups.tf  
│   ├── ec2.tf, inventory.tpl  
│   ├── alb.tf, monitoring.tf  
│   ├── outputs.tf  
│   └── terraform.tfvars.example  
└── ansible/  
    ├── ansible.cfg, site.yml  
    ├── inventory.ini (generated)  
    ├── group_vars/all.yml  
    └── roles/  
        ├── nginx/  
        └── websites/
```

terraform/

What exists in AWS: network, servers, load balancer, alarms

ansible/

How the servers are set up: Nginx, proxy rules, websites

group_vars/all.yml

Add a new website by adding one entry

HOW THE REVERSE PROXY WORKS



URL path	Backend	Website
/employee	127.0.0.1:8001	Employee
/customer	127.0.0.1:8002	Customer
/insurance	127.0.0.1:8003	Insurance
/reports	127.0.0.1:8004	Reports
/support	127.0.0.1:8005	Support
/	Landing page	Hub

The trailing slash in proxy_pass removes the /insurance/ prefix, so the backend serves its own index page.

- 1 Browser opens /insurance/
- 2 ALB picks a healthy EC2 server
- 3 Nginx matches location /insurance/
- 4 proxy_pass sends it to 127.0.0.1:8003
- 5 Response returns through Nginx and the ALB

RESULT



Three failure drills show each layer of protection working.

SERVER OUTAGE

Stop Nginx on one server

The ALB marks it unhealthy and sends all traffic to the other server. CloudWatch raises the alarm.

NGINX CRASH

`pkill -9 nginx`

The process is recovered automatically. Confirm with `systemctl status nginx`.

CONFIGURATION DRIFT

Delete the proxy config

Re-running `ansible-playbook site.yml` restores the correct configuration.

ANALYSIS: PRODUCTION VALUE



No single point of failure

Two servers in two zones behind an ALB keep serving if one fails.



Detection and alerting

Health checks remove bad servers and alarms notify the team.



Repeatable environments

Terraform recreates dev, test or recovery copies on demand.



Consistent servers

Ansible gives every server one configuration and repairs drift.



Easy to extend

A new website is one entry in group_vars/all.yml.



Smaller attack surface

Web traffic only via the ALB, SSH only from one IP.

HINDSIGHT AND NEXT STEPS



WHAT WORKED WELL

- Terraform for infrastructure and Ansible for configuration kept each tool simple.
- One Nginx tier gave path-based routing for five sites with a small security surface.
- Failure drills made the value of health checks and re-runnable automation visible.

WHAT COMES NEXT

HTTPS	Add a certificate and redirect HTTP
Auto Scaling	Capacity follows demand
Private subnets	Only the ALB stays public
Remote state	Safe Terraform for teams
CI/CD	validate and plan on every change

CONCLUSION



Built as code

Terraform creates the network, servers, load balancer and alarms.



Configured by playbook

Ansible turns blank servers into an Nginx reverse proxy for five sites.



Watched and recoverable

CloudWatch and the ALB detect failures, and Ansible repairs drift.



THANK YOU

Questions?